arXiv:2304.10618v1 [cs.AR] 20 Apr 2023

ULEEN: A Novel Architecture for Ultra Low-Energy Edge Neural Networks

Zachary Susskind, Aman Arora
The University of Texas at Austin
Igor Dantas Dos Santos Miranda
Federal University of Recôncavo da Bahia
Alan Tendler Leibel Bacellar, Luis Armando Quintanilla Villon, Rafael Fontella Katopodis
Federal University of Rio de Janeiro
Leandro Santiago de Araújo
Universidade Federal Fluminense
Diego Leonel Cadette Dutra, Priscila Machado Vieira Lima
Federal University of Rio de Janeiro
Felipe Maia Galvão França
Instituto de Telecomunicações, Porto
Federal University of Rio de Janeiro
Mauricio Breternitz Jr.
ISCTE - Instituto Universitario de Lisboa
Lizy K. John
The University of Texas at Austin

***Abstract*—The deployment of AI models on low-power, real-time edge devices requires accelerators for which energy, latency, and area are all first-order concerns. There are many approaches to enabling deep neural networks (DNNs) in this domain, including pruning, quantization, compression, and binary neural networks (BNNs), but with the emergence of the "extreme edge", there is now a demand for even more efficient models. In order to meet the constraints of ultra-low-energy devices, we propose ULEEN, a model architecture based on weightless neural networks.**

Weightless neural networks (WNNs) are a class of neural model which use table lookups, not arithmetic, to perform computation. The elimination of energy-intensive arithmetic operations makes WNNs theoretically well suited for edge inference; however, they have historically suffered from poor accuracy and excessive memory usage. ULEEN incorporates algorithmic improvements and a novel training strategy inspired by BNNs to make significant strides in improving accuracy and reducing model size.

We compare FPGA and ASIC implementations of an inference accelerator for ULEEN against edge-optimized DNN and BNN devices. On a Xilinx Zynq Z-7045 FPGA, we demonstrate classification on the MNIST dataset at 14.3 million inferences per second (13 million inferences/Joule) with 0.21 μs latency and 96.2% accuracy, while Xilinx FINN achieves 12.3 million inferences per second (1.69 million inferences/Joule) with 0.31 μs latency and 95.83% accuracy. In a 45nm ASIC, we achieve 5.1 million inferences/Joule and 38.5 million inferences/second at 98.46% accuracy, while a quantized Bit Fusion model achieves 9230 inferences/Joule and 19,100 inferences/second at 99.35% accuracy. In our search for ever more efficient edge devices, ULEEN shows that WNNs are deserving of consideration.

I. Introduction

In the last decade, deep neural networks (DNNs) have driven revolutionary improvements in domains including image classification, natural language processing (NLP), and medical diagnostics, in some cases achieving superhuman accuracy [1]. This advancement has been driven by an exponential increase in the size and complexity of the models we can train. At the same time, there is a second movement in the opposite direction: deployment of AI models on *smaller and smaller devices*. Techniques such as pruning, compression, low-precision quantization [2], and Once-for-All [3] can greatly reduce the memory requirements and computational demands of large models, enabling their use on low-power, resource-constrained edge devices. However, on the "extreme edge", where computation is performed directly adjacent to physical sensors [4], energy budgets are minuscule and efficiency is of utmost importance. Such devices are often deployed in inaccessible environments, and may be expected to last years or decades on a small battery, or by harvesting energy directly from the phenomena they measure [5].

Binary neural networks (BNNs) [6], [7], [8], [9], [10] take quantization to its logical extreme, reducing network weights and activations to single-bit values. By replacing energy-intensive multiplication with XNOR operations, they achieve energy efficiency orders of magnitude better than DNNs. However, like DNNs, BNNs must propagate activations through many layers of computation. This may still present a significant critical path for real-time edge applications.

Weightless Neural Networks (WNNs) are a distinct class

of neural model which perform computation primarily using lookup tables, rather than arithmetic or logical functions. Individual weightless neurons, also known as RAM nodes, concatenate binary inputs to form an address into their lookup table, and produce a binary response. WNNs are inspired by the dendritic trees of biological neurons, which perform highly nonlinear decode processing [11]. Unlike traditional or binary neurons, RAM nodes are capable of learning *nonlinear* functions of their inputs.

The concept of WNNs is not new - the earliest implementations date to the 1950s [12]. Their simple structure was well-suited to early VLSI techniques, and they achieved some prominence in the 1970s-80s. However, these early WNNs were surpassed by DNNs in both accuracy and memory efficiency by the 1990s. Thus, while the simplicity and non-linearity of WNNs makes them appealing for deployment on the edge, algorithmic and architectural enhancements are needed to make them comparable or superior to optimized DNN and BNN models.

In this paper, we demonstrate techniques to improve the accuracy and reduce the hardware requirements of WNNs. We present a weightless neural architecture we call ULEEN - **U**ltra **L**ow **E**nergy **E**dge **N**etworks, suitable for inference applications under extreme energy constraints. ULEEN incorporates a set of techniques not previously used in WNNs, including efficient ensembles, pruning, and a multi-epoch gradient-based training algorithm using the straight-through estimator [13]. We also improve upon prior work by including counting Bloom filters with hardware-friendly hashing, a novel nonlinear thermometer encoding, and threshold-based one-shot training [14], [15]. We present FPGA and ASIC implementations of an inference accelerator for this architecture and compare it against state-of-the-art efficient inference platforms for DNNs and BNNs.

Our specific contributions in this paper are as follows:

1) ULEEN, a weightless neural architecture which incorporates principles from prior WNNs, BNN-inspired training methodologies, and further algorithmic enhancements. Through these improvements, we increase the best reported WNN accuracy on the MNIST dataset from 91.5% to 98.46%.
2) An energy-efficient inference accelerator architecture for ULEEN which can be implemented on an FPGA or as an ASIC.
3) Comparisons of our ULEEN accelerator with state-of-the-art quantized, optimized DNNs and BNNs on ASIC and FPGA. In comparison to FINN [8] (FPGA), we demonstrate 1.4-2.6x improved latency, 1.2-2.6x improved throughput, and 6.8-8.5x improved energy at equal or better accuracy. In comparison to Bit Fusion [16] (ASIC), we demonstrate 2014-19549x improved throughput and 479-663x improved energy with ¡1% reduction in accuracy.
4) A toolchain for generating ULEEN models using either single- or multi-shot training rules. A second toolchain which produces RTL from trained ULEEN models using our accelerator architecture. These are available at: *URL omitted for double-blinding*.

The remainder of our paper is organized as follows: In Section II, we provide additional background on WNNs and one specific WNN, WiSARD, which we use as the base model for our improvements. In Section III, we present the ULEEN architecture in detail. In Section IV, we discuss software and hardware implementation details. In Section V, we compare our accelerator architecture against prior DNN and BNN accelerators and our model architecture against prior memory-efficient WNNs, and provide additional results exploring the performance of our models. In Section VI, we give some additional context into the prior work in this domain. Lastly, in Section VII, we discuss future work and conclude.

II. Background

Weightless neural networks (WNNs) are neural models which perform computation using lookup tables. The basic computational unit, the RAM node, is an n-input, 2^n-output lookup table with learned 1-bit entries. Each permutation of the contents of this table represents a unique Boolean function, meaning there are 2^{2^n} possible functions for a single RAM node, many of which are nonlinear. By contrast, a single `XNOR`-and-popcount neuron in a BNN can only learn one of the $n * 2^n$ linear functions of its inputs.

The downside of this expressiveness is that the size of a RAM node grows exponentially with its number of inputs, quickly becoming intractable. Therefore, layers in a WNN are typically composed of many RAM nodes which are each only sensitive to a subset of the layer's inputs.

Training a WNN entails learning Boolean functions in its component RAM nodes. Many approaches have been explored for this, including both supervised [17] and unsupervised [18] techniques. Most typically, WNNs are trained using a supervised one-shot approach. All RAM nodes begin filled with zeros. Encoded inputs are presented sequentially to the network. When a RAM node receives a training input, it sets the corresponding location in its memory to 1. Note that presenting the same pattern to a node again has no further effect; thus, there is no advantage to having multiple epochs of training. By leveraging one-shot training techniques, WNNs can be trained up to four orders of magnitude faster than DNNs and other well-known computational intelligence models such as SVM [19].

Since WNNs only learn patterns they were exposed to during training, one might expect them to have difficulty generalizing to new data. However, although an inference sample may not be identical to any training sample, many of the "subpatterns" seen by individual RAM nodes will have also been present in training data. Therefore, as long as an inference sample is not too different from any training sample, the network can still effectively generalize.

We have only described the fundamentals of WNNs here; many variants have been explored, and we refer the curious reader to [12] for a more in-depth discussion. We will however present one WNN model architecture, WiSARD, in detail, since it serves as the baseline for the improvements discussed in this paper.

Though not the focus of this paper, computer architects may note some similarities between WNNs and the predictors used in modern microprocessors. For instance, RAM nodes are conceptually similar to branch predictor tables, and using a concatenated input vector to index into a RAM node is conceptually similar to using a branch history register in a table-based branch predictor. Table updates in branch predictors can be viewed as an online version of the one-shot training rule.

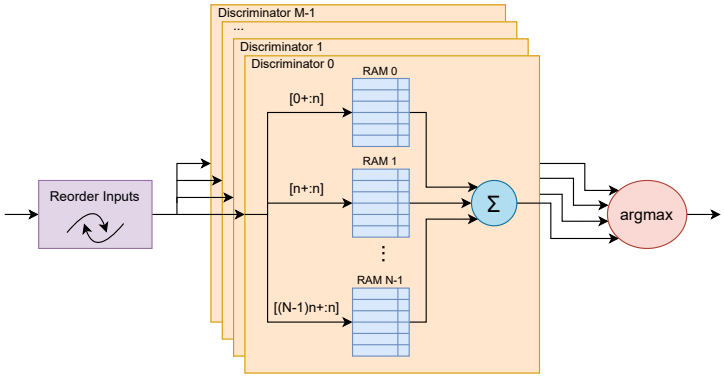


Fig. 1. A depiction of the WiSARD WNN model with M classes, n inputs per RAM node, and N RAM nodes per discriminator. This model has a total of MN RAM nodes and $MN2^n$ bits of state.

WiSARD: While foundational research in WNNs began in the 1950s, the first WNN architecture to achieve broad success was WiSARD (Wilkie, Stonham, and Aleksander's Recognition Device) [20], introduced in 1981 and produced commercially from 1984. WiSARD was designed for classification tasks for purposes such as anomaly detection. A WiSARD model, as depicted in Figure 1, is composed of submodels, known as *discriminators*, which are each specialized for one of the possible output classes. These discriminators are in turn composed of n-input RAM nodes; for an I-input model, there are $N \equiv I/n$ such nodes per discriminator. Inputs to the network are assigned to RAM nodes using a pseudo-random mapping (also referred to as reordering); typically this same mapping is shared between all discriminators, meaning RAM nodes at the same index in different discriminators will have the same inputs.

During training, inputs are presented only to the discriminator corresponding to the correct output class. During inference, inputs are first presented to all discriminators. The outputs of the RAM nodes in each discriminator are then summed to produce response values, and the class corresponding to the discriminator with the strongest response is taken to be the prediction of the network. Figure 2 shows a simplified WiSARD model performing inference. In this example, the response from Discriminator 1 is the strongest since the input image contains the digit "1".

If an input seen during inference is identical to one seen during training, all RAM nodes of the corresponding discriminator will output 1, yielding the maximum possible response. On the other hand, if a pattern is similar but not identical to training patterns, then some subset of the RAM nodes may produce a 0, but many will still output 1. As long as the response of the correct discriminator is still stronger than the responses of any other discriminator, the network will output a correct prediction. In practice, WiSARD has a far greater ability to generalize than simpler WNN models.

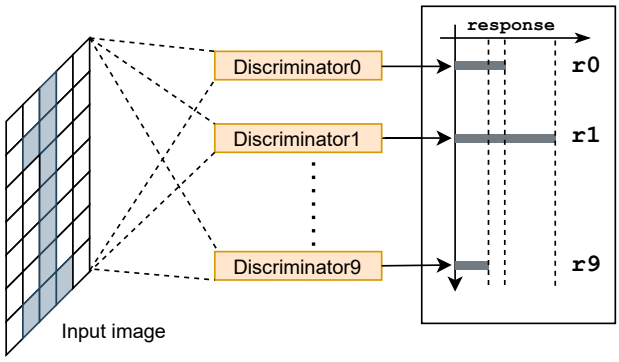


Fig. 2. A WiSARD WNN model recognizing digits. In this example, the input image contains "1", and the corresponding discriminator produces the strongest response.

WiSARD's performance is directly related to the choice of n. Small values of n force the model to only learn simple patterns, which may improve its ability to generalize but can also prevent it from effectively learning. Large values of n produce more specialized behavior, but may also result in overfitting to training data [20]. Recent results have formally demonstrated that the VC dimension[1] of WiSARD is very large [22], indicating it has a large theoretical capacity to learn patterns.

III. Proposed Design: ULEEN

The ULEEN model includes several enhancements over the baseline WiSARD model: (i) counting/continuous Bloom filters, (ii) non-linear thermometer encoding, (iii) ensembles, and (iv) pruning. Furthermore, ULEEN models can be trained in two ways: (i) an enhanced version of the conventional one-shot technique or (ii) a novel multi-shot training algorithm, inspired by BNNs and based around backpropagation and the straight-through estimator.

A. Model Overview

Figure 3 shows the ULEEN model at a high level, including training with the multi-shot technique. Inputs are fed to an ensemble of smaller models. We find that ensembles of small models give better accuracy than monolithic large models, and incorporate this into ULEEN's design. The aggregation of the results from individual models is done in the "Vectorized Addition" block in the figure. During multi-shot training, we

[1]The Vapnik–Chervonenkis (VC) dimension measures the complexity of the knowledge represented by a set of functions that can be encoded by a binary classification algorithm [21]. While usually approximated by statistical methods, it is possible to establish the exact VC dimension for some learning methods, including WiSARD.

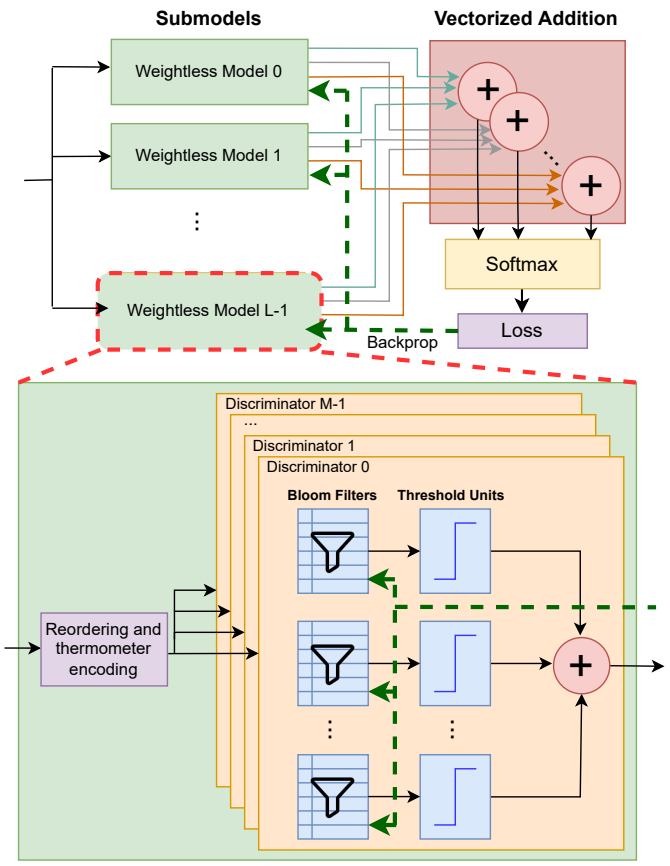


Fig. 3. The ULEEN model is composed of an ensemble of submodels, each of which is a WNN. Each submodel is composed of discriminators. Discriminators use continuous Bloom filters during training to allow for gradient-based weight updates. This figure shows the multi-shot training process, which uses backpropagation based on the straight-through estimator.

take a softmax of the outputs of the aggregation and compute cross-entropy loss.

A multi-bit thermometer encoding is used to represent inputs with greater granularity. RAM nodes are implemented using hash-based Bloom filters for compression. During multi-shot training, Bloom filters internally hold floating-point values and are binarized using a unit step function.

The multi-shot training technique for ULEEN is a modified version of backpropagation based on the straight-through estimator [13]. The flow of gradients during backpropagation is shown with the green dotted arrow in the figure; the threshold units are treated as identity functions and ignored. Pruning, which eliminates RAM nodes which contribute least to overall accuracy, is a post-training technique and is therefore not shown in Figure 3.

1) Counting/Continuous Bloom Filters: A major challenge impacting WiSARD is its exponential growth in model size as the number of inputs to each RAM node increases. However, in practice, these large RAM nodes learn Boolean functions with few minterms, meaning their contents are sparse. In Bloom WiSARD [23], it was demonstrated that hash-based data structures such as Bloom filters can be used to reduce the data footprint of RAM nodes. Bloom filters consist of a lookup table and k independent hash functions, and output 1 only if all k hashed locations corresponding to an input are 1. Since no collision checking occurs, Bloom filters may produce false positives. However, this was found to have minimal impact on accuracy.

In order enable more sophisticated training approaches, we use two variants of the Bloom filter: *counting* and *continuous* Bloom filters. After training, the contents of these filters are binarized, and they are replaced with conventional Bloom filters for inference. Figure 3 shows the substitution of enhanced Bloom filters in place of conventional RAM nodes in the discriminators.

When training a WiSARD model using conventional Bloom filters, RAM entries are set the first time a pattern is seen during training, and are not subsequently updated. The issue with this approach is that it treats rare patterns as equally important to common ones. Bleaching [15] is a technique in which patterns which were seen fewer than some threshold b times during training are ignored during inference. Typically, this is achieved using tables of counters. In ULEEN, while training using the one-shot strategy, we replace Bloom filters with counting Bloom filters, which use multi-bit counters instead of single-bit entries, enabling bleaching and hashing in the same model. During training, whenever a pattern is seen, the smallest of its corresponding counter values is incremented (multiple counters in the event of a tie). Figure 4 shows the behavior of a counting Bloom filter during inference. The input to the filter is hashed, and these hashes are used to access counter values. For input x_1, this corresponds to counter values $\{3, 4, 2\}$; for x_2, this corresponds to $\{3, 4, 3\}$. The smallest accessed counter value is then found (2 for x_1; 3 for x_2) and compared against a threshold value b to determine the response of the filter. For this example, the threshold $b = 3$, so x_1 produces an output of 0 and x_2 an output of 1. Filter responses become "possibly seen at least b times" and "definitely not seen b times".

When training using the multi-shot strategy, we go further by replacing Bloom filter entries with floating point values. These *continuous* Bloom filters produce an output of 1 if the smallest entry is at least 0. Using continuous Bloom filters allows us to train using gradient-based techniques.

Since the hash functions used for our Bloom filters do not need to be cryptographically secure and have constant-length input, we are able to use simple **arithmetic-free** hash functions. In particular, we use functions drawn randomly from the H3 family [24], a set of hash functions which consist of simple logical operations and differ from each other only by the choice of a random parameter. Bloom WiSARD derived its independent hash functions using a double-hashing technique based on the MurmurHash [25] algorithm. Though simpler than cryptographic hash functions, MurmurHash is designed to handle variable-length inputs and requires substantial arithmetic.

2) Gaussian Nonlinear Thermometer Encoding: Inputs to WNNs are traditionally represented as 1-bit values: 1 if an input is greater than its mean in the training data and 0 otherwise.

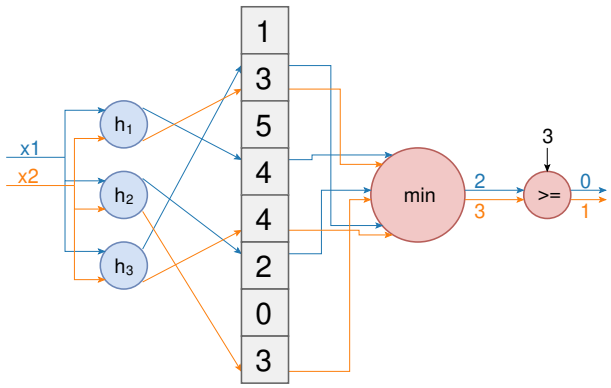


Fig. 4. An example of a counting Bloom filter with three hash functions and threshold $b = 3$.

However, the granularity that can be provided using a single bit is very limited. Integer encodings are not a good choice, since different bits carry different amounts of information, meaning the least significant bits are essentially noise when used to form an address. Thermometer encodings are the preferred multi-bit encoding in prior work [26].

Thermometer encoding is a *unary* coding in which a value is compared against a series of increasing thresholds, with the i'th bit of the result representing the comparison against the i'th threshold. As Figure 5 shows, input values behave like the mercury in an analog thermometer: as a value increases and passes thresholds, bits are set from least to most significant.

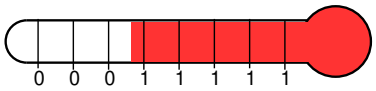


Fig. 5. Like the mercury passing the gradations in a thermometer, in a thermometer encoding, bits are set to 1 from least to most significant as the encoded value increases.

Most prior work using thermometer encodings uses equal intervals between the thresholds. The disadvantage of this approach is that a large number of bits may be dedicated to encoding outlying values. In ULEEN, we instead assume that each input follows a normal distribution, and compute its mean and standard deviation from training data. For a t-bit encoding, we use thresholds to divide the Gaussian into $t+1$ regions of equal probability. This Gaussian encoding provides increased resolution for values near the center of their range. Gaussian thermometer encodings were also explored in [27] in a narrow scope; we use them more generally, showing they are useful even when the underlying data is not actually Gaussian.

3) Efficient Ensembles: Ensembles, as shown in Figure 6, combine multiple weak classifiers into a single strong classifier. Ensembles have been extensively studied in machine learning, and are the driving concept behind machine learning techniques such as Bayesian averaging, Boosting, and Bagging [28].

One consideration when designing an ensemble model is how the predictions of multiple submodels should be combined into a single output. As Figure 3 shows, ULEEN composes

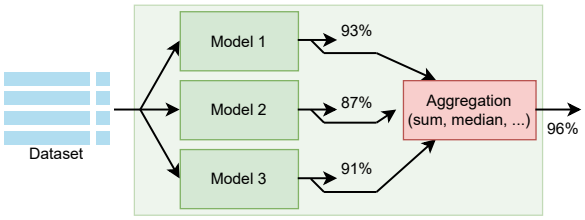


Fig. 6. Simplified view of an ensemble model. An ensemble combines multiple weak classifiers to create a single stronger classifier.

submodels by summing the response values for each discriminator across the submodels. In other words, a model with L submodels and M classes will produce an $L \times M$ matrix of responses R, and a top-level M-element response vector $\vec{r} \triangleq R^{\intercal}\vec{1}$.

One might reasonably expect that using ensembles would increase total model size, since the number of RAM nodes is much greater. In practice, we have found this to not necessarily be the case - recall that the key strength of ensembles is their ability to combine weak classifiers. This means that individual submodels of an ensemble can be made much smaller than standalone models without harming ensemble accuracy.

In an ensemble of submodels with different numbers of inputs per filter, filters with many inputs may also selectively "ignore" simple patterns where most inputs are don't-cares, relying on filters with fewer inputs to capture these patterns. This helps reduce the number of table entries needed for Bloom filters.

Since the amount of hashing required for inference increases linearly with the number of submodels, we avoid using ensembles with many submodels. We also do not train ensembles with the one-shot training rule, since this rule gives poor accuracy when small Bloom filters have many inputs.

Ensembles of WiSARD models were investigated in [29], but without backpropagation or multi-shot training, and resulted in large models with marginal accuracy benefit.

4) Pruning: Pruning removes parameters and connections from a model to reduce its size and complexity. Optimized DNNs [3], [30], [31], [32], [33] have applied pruning techniques with excellent results, but similar concepts have not been used in prior WNNs.

After training, the correlations between RAM node outputs and correct model outputs are calculated for each RAM node in the model. A fixed fraction of the RAM nodes with the lowest correlation in each discriminator are removed from the model. Since this pruning reduces the maximum possible response of a discriminator, we next learn a set of integer biases which are added to the outputs of the discriminators. Finally, we fine-tune the remaining RAM nodes using the multi-shot training process.

In our experiments, we observed that we could typically prune around 30% of RAM nodes, reducing model size proportionately, with a minimal impact on accuracy. Though this does not approach the degree of pruning which is frequently possible for DNNs, it is still a significant reduction in ULEEN's memory requirement. When used in an ensemble, the bias can

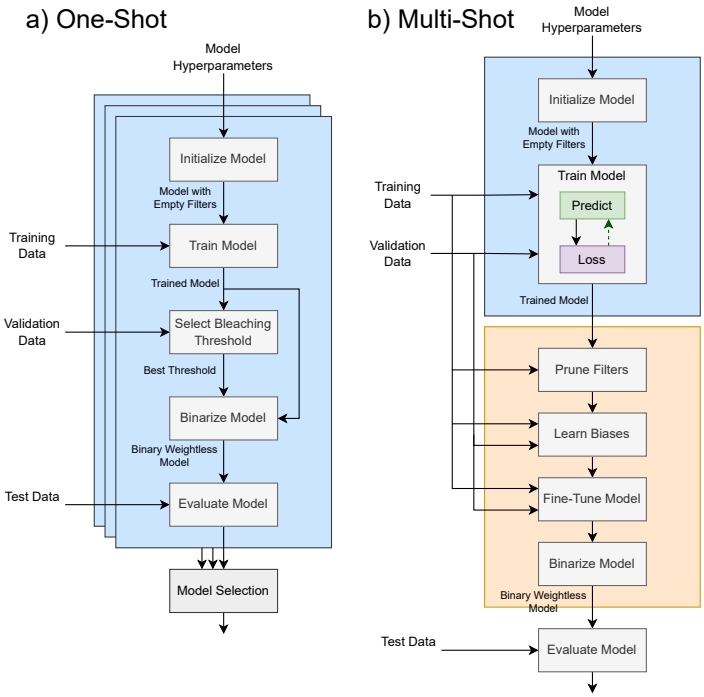


Fig. 7. ULEEN supports both one- and multi-shot training techniques. (a) In one-shot training, encoded training samples are sequentially presented to the model and used to update counter values. Afterwards, a bleaching threshold is optimized to eliminate rare patterns. (b) In multi-shot training, a gradient-based update rule is used to learn counter values over multiple epochs. Next, weakly correlated filters are pruned and replaced with a constant bias, and the remaining filters are fine-tuned.

be summed across the submodels, meaning the only overhead for pruning is a single addition.

B. Training ULEEN

We use both one-shot and multi-shot training techniques for ULEEN. The one-shot technique is similar to how WiSARD models are traditionally trained. The multi-shot technique provides better accuracy, particularly for larger datasets, and is necessary in order to use ensemble models and pruning, but does require much more training time than the one-shot approach.

1) One-shot Training: The one-shot training technique is a computationally simple approach which is similar to how prior WNNs were trained. The process is summarized in Figure 7a. Hyperparameters include the thermometer encoding and the configuration information for the Bloom filters (number of inputs, table size, and number of hash functions). During training, encoded input samples are sequentially presented to the discriminator corresponding to the correct output class. Counting Bloom filters update their contents by incrementing counters according to the method discussed previously (Section III-A1).

After training, patterns that were rarely seen during training are discarded using bleaching [15]. Bleaching finds a value b such that all patterns seen fewer than b times during training are discarded. If bleaching is not used, almost all RAM node entries may become 1 given a large training dataset, harming generalization. This behavior is known as *saturation*. WNNs which do not use bleaching need careful manual feature extraction and data selection to avoid saturation.

We use a binary search strategy to find the value of b which maximizes model accuracy on the validation set. Afterwards, all filter values less than b are replaced with 0, and all remaining filter values are replaced with 1. This allows the counting Bloom filters to be replaced with conventional binary Bloom filters for inference, simplifying the hardware.

2) Multi-shot Training: While the one-shot technique is efficient, it is ultimately limited by its inability to consider feedback during training. However, conventional gradient-based techniques do not work for WNNs since the outputs of Bloom filters are binary rather than continuous. A similar problem also impacts BNNs, which led to the development of gradient-based training algorithms that worked with binary weights. We base our multi-shot training technique on the BNN training algorithm described in [7].

ULEEN uses continuous Bloom filters, with floating-point entries between -1.0 and 1.0, for training with the multi-shot technique. During the forward training pass, entries are binarized using a unit step function:

$$f(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

Backpropagation does not work with the unit step function, since its derivative is 0 everywhere except $x = 0$ (where it is infinite). However, when computing gradients, we instead treat the unit step function as the identity function $f(x) = x$, meaning $f'(x) = 1$. This technique, known as the *straight-through estimator*, has been used for training low-precision and binary networks [13], but we are the first to use it for WNNs. Since our model is only a single layer, we do not have to propagate gradients through the indexing and hash operations.

Figure 7b summarizes our multi-shot training process. After training, models are pruned and fine-tuned according to the method discussed in Section III-A4. After pruning, the continuous Bloom filters are binarized by applying the unit step function.

Models were trained using the Adam optimizer [34] with base learning rate 10^{-3}. Weights were initialized with uniform distribution $\mathscr{U}(-1, 1)$. To prevent overfitting, we added dropout regularization [35] ($p = 0.5$) to the outputs of the filters. For the MNIST [36] dataset, we also experimented with a simple form of data augmentation: we made 9 copies of each image in the training set, shifted horizontally and vertically between -1 and 1 pixels.

C. Inference Accelerator Architecture

Figures 8 and 9 show the block diagram of our pipelined ULEEN inference accelerator. In order to simplify control logic, units on the chip operate in lockstep to the greatest extent possible. This means that an entire input sample must be read in before computation can begin. This deserialization is performed by the bus interface (not shown in the figure). Input data may optionally be compressed by replacing unary thermometer-encoded values with binary values representing how many bits are set. This reduces data movement from

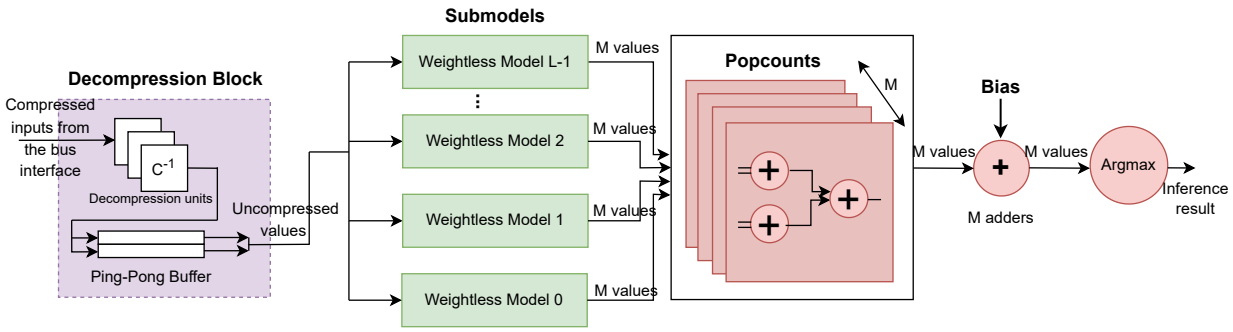


Fig. 8. A diagram of the ULEEN inference accelerator architecture. Input is deserialized and, if needed, decompressed, before being passed to an ensemble of submodels. The outputs of the submodels are summed and biased to get per-class responses, and the index of the strongest response is taken as the prediction.

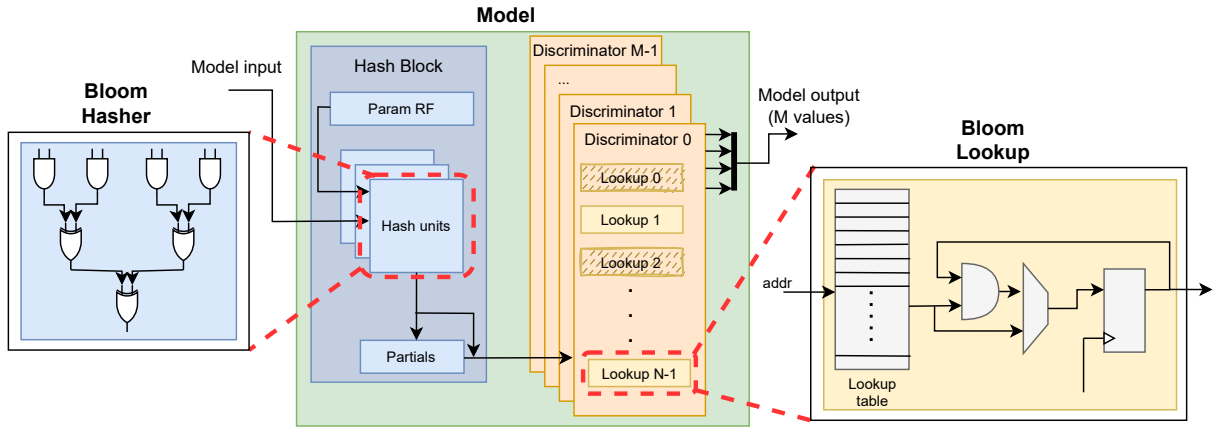


Fig. 9. Details of each submodel in the ULEEN inference accelerator. Each submodel contains a hardware block for computing hash functions, and a number of hardware units for performing lookups. These blocks collectively compose the Bloom filters, which are divided into separate units to eliminate redundant computation.

off-chip, but requires a decompression unit to recover the thermometer encoding, shown on the left in Figure 8. This unit is eliminated from the design if input data is not compressed.

The discriminators in Figure 9 exhibit several optimizations. Though each H3 hash function in a Bloom filter requires a different random parameter, there is no disadvantage to sharing these parameters between all Bloom filters in a submodel. Therefore, a central register file (shown as "Param RF" in the figure) is used to store hash parameters. Since input order is shared between discriminators in a submodel, when hash parameters are also shared, all discriminators will receive the same hashed values. Therefore, it is redundant to calculate hashes for each discriminator separately; instead, we use a single central hashing block. The hash block is itself composed of many pipelined hash units which process input sequences with a throughput of 1 hash/cycle. As shown in the left part of Figure 9, these hash units perform only AND and OR operations, with no arithmetic component. If hashing is fully parallelized, the performance of the design will be heavily bottlenecked by off-chip bandwidth. Therefore, we reduce the number of hash units to the minimum number sufficient to achieve maximum throughput, and accumulate partial hash results in a buffer. Once the buffer is full and the last partial hash is available, all Bloom filters perform a lookup in lockstep.

Since hashing is moved into a central block, discriminators themselves contain only lookup units. The lookup unit, shown at the right of Figure 9, consists of a lookup table and the hardware to perform an AND reduction. A 1-bit accumulator in each lookup unit can take as input either the output of the LUT or the AND of that output and its current contents. Once all hash lookups have been performed, the outputs of the lookup units are marked as valid.

Each submodel in an ensemble must compute its own hashes, since input orders and hash input and output widths vary. Since different submodels have different table contents, sizes, and pruning, they also have their own sets of filter units.

Popcounts and submodel response accumulation are performed using a vector of adder trees, shown in the right of Figure 8. Bias values are added to these results (Section III-A4), and the index of the strongest response is computed to produce a final inference result.

IV. Evaluation Methodology

We compare ULEEN against state-of-the-art BNNs, quantized DNNs, and WNNs, using designs in FPGAs, ASICs and

software models. The methodology for the various comparisons is described in this section.

The FINN [8], [37] framework from Xilinx Research Labs for BNN inference on FPGAs is used as a comparison for our FPGA implementation. We use the SFC-max, MFC-max and LFC-max networks described in [8]. These are 3-layer fully-connected network topologies for classifying the MNIST dataset [36], with different numbers of neurons (SFC contains 256 neurons per layer, MFC contains 512 neurons per layer and LFC contains 1024 neurons per layer). The "max" suffix indicates that these are performance-optimized designs intended to achieve high peak throughput. To the best of our knowledge, these are most efficient implementations of BNNs on FPGAs for MNIST.

We compare our ASIC implementation against Bit Fusion [16], [38], a quantized DNN hardware architecture which supports dynamic precision. A highly quantized LeNet-5 architecture (2-bit; Ternary) is used by Bit Fusion for classifying MNIST. This NN architecture is proposed in [39] and shown to have accuracy close to the original LeNet5. We use a Bit Fusion design with precision optimized for this model; i.e. minimum and maximum precision are 2 bits. We identify three efficient variations of the accelerator for the workload, with configuration parameters:

{S=8x8, WBUF=32KB, ABUF=16KB, OBUF=8KB}
{S=16x16, WBUF=64KB, ABUF=32KB OBUF=16KB}
{S=32x32, WBUF=64KB, ABUF=32KB, OBUF=16KB}

Here, 'S' denotes the size of the systolic array in the Bit Fusion core, and WBUF, ABUF, OBUF denote the size of the weight, activation and output buffers respectively. We refer to these variations as BF8, BF16 and BF32 respectively in Section V. To implement the Bit Fusion accelerator, we obtained the RTL from the authors [16] and synthesized it using Cadence RTL Compiler 2017 and the FreePDK45 [40] library. The Bit Fusion simulator [38], which combines the energy for the core reported by Cadence with the energy for RAMs/buffers reported by Cacti [41], was used to evaluate the total energy spent on inference.

In order to gauge the impact of our improvements versus prior WNNs, we compared ULEEN model sizes and accuracies against Bloom WiSARD [23] on the same nine multi-class classification datasets originally used to evaluate that model. For datasets which did not have explicit train and test sets, we performed a 2:1 train/test split.

A. Software Implementation

We implemented one-shot training in Python, using the Numba JIT compiler to optimize performance-critical sections. This code is CPU-only and single-threaded, but we used a machine with 64 cores (2x Intel E5-2698 v3) to train many models in parallel. We used a semi-automated sweeping methodology to identify optimal model hyperparameters. In particular, we swept over ranges of values for inputs, entries, and hash functions per Bloom filter and thermometer encoding bits per input. The results of this sweep were then used to identify models which achieved a good balance between accuracy and size.

The multi-shot training flow is considerably more computationally intensive, as it requires gradient computation in addition to multiple epochs of training. However, it provides increased accuracy, and is necessary to use ensembles and pruning. We implemented multi-shot training using the PyTorch machine learning framework, and ran training on an NVIDIA Tesla M40 GPU. Forward and backward passes for Bloom filters were implemented as a single multi-dimensional gather/scatter operation, enabling efficient memory-parallel computation. Despite these optimizations, training times were much longer than with the one-shot algorithm. We believe this algorithm could benefit greatly from the much higher memory bandwidth and larger caches available on newer GPUs.

B. Hardware Implementation

The ULEEN accelerator source is written using Mako-templated SystemVerilog. Mako [42] is a template library which allows the Python interpreter to be run as a preprocessing step for code in arbitrary languages. We used Mako to automatically extract parameters from trained models, construct state machines, and decide how many functional units to instantiate in order to hit throughput targets while minimizing area. This templating allowed us to generate new accelerators just by changing command-line parameters. We simulated our designs using Synopsys VCS 2018 to ensure functional correctness and to evaluate the latencies and throughputs of our accelerators.

For our FPGA implementation and comparison, we use the Zynq Z7045 SoC platform, which was also used for FINN. Our design has the same I/O interface width as FINN (112 bits). We used Xilinx Vivado 2019.2 to synthesize and implement our designs. We targeted the same frequency of operation (200 MHz) as FINN, though we were unable to achieve this frequency in all cases due to FPGA routing congestion. Resource usage and power consumption were obtained from Vivado reports.

For the ASIC implementation and comparison, we use the same tool and library as Bit Fusion for a fair comparison (FreePDK45 [40] and Cadence RTL Compiler 2017). Our designs also have the same 192-bit interface width. The frequency of operation is 500 MHz, the same as the Bit Fusion designs. We used area and power numbers reported by Cadence RTL Compiler (there are no RAM blocks in our design, so using Cacti is not required, unlike with Bit Fusion).

V. Results

A. Selected ULEEN Models

We identified three ULEEN models (ULN-S, ULN-M, ULN-L), shown in Table I, which target different design points along the size/accuracy curve. All submodels use two hash functions per filter - we found that using one hash function was not always sufficient to avoid excessive collisions, while more than two hash functions increased hardware cost with no clear benefit to accuracy. All submodels within a model use the

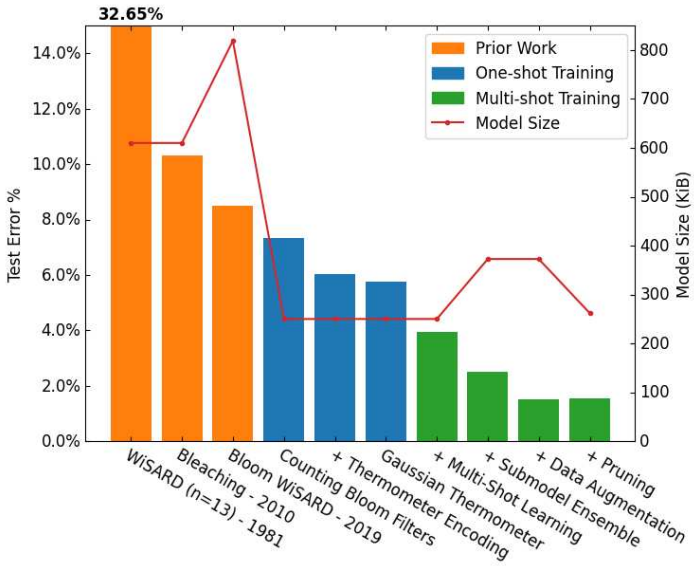


Fig. 10. Iterative impacts of our improvements to WNNs. We show three prior results, including the previous best model (Bloom WiSARD), and the improvements to error and model size as we sequentially incorporate ULEEN's optimizations. Results for MNIST dataset.

same thermometer encoding, and all models were pruned at a 30% ratio. As shown in the table, the ensembles consisted of between 3 and 6 submodels, with ensemble accuracies ranging from 96.20% to 98.46%. Some of the individual submodels have accuracies as low as 80%; however, as discussed in Section III-A3, ensembles are able to combine weak classifiers to produce accurate predictions.

TABLE I
DETAILS OF THE SELECTED ULEEN MODELS. SMX REFERS TO INDIVIDUAL SUB MODELS COMPRISING THE ENSEMBLE MODEL.

Model	Sub-Model	Bits /Inp	Inputs /Filter	Entries /Filter	Size (KiB)	Test Acc.%
ULN-S	Ensemble	2	{}	{}	16.9	96.20
	SM0	2	12	64	7.19	92.91
	SM1	2	16	64	5.39	90.25
	SM2	2	20	64	4.38	86.16
ULN-M	Ensemble	3	{}	{}	101	97.79
	SM0	3	12	64	10.9	83.54
	SM1	3	16	128	16.0	90.93
	SM2	3	20	256	26.0	92.92
	SM3	3	28	256	18.44	87.05
	SM4	3	36	512	29.38	80.93
ULN-L	Ensemble	7	{}	{}	262	98.46
	SM0	7	12	64	25.0	88.78
	SM1	7	16	128	37.7	93.24
	SM2	7	20	128	30.2	92.44
	SM3	7	24	256	50.3	93.92
	SM4	7	28	256	43.1	90.47
	SM5	7	32	512	75.6	90.44

B. Performance of ULEEN enhancements

Figure 10 illustrates the impacts of the optimizations we use in ULEEN. The first three models are prior work, including the original 1981 WiSARD architecture [20] and the state-of-the-art 2019 Bloom WiSARD architecture [23]. Subsequent models incorporate progressively more of our improvements. Our final model, ULN-L, reduces error by 82% and model size by 68% versus Bloom WiSARD. Using multi-shot learning and ensemble models yielded significant improvements to accuracy, while pruning provides a significant reduction in model size.

C. FPGA Comparison (vs FINN)

Table II and Figure 11 show the comparison of ULEEN with FINN. ULN-S, ULN-M, ULN-L are compared to the FINN SFC, MFC and LFC models respectively. We report accelerator latency in μs and peak throughput in thousands of inferences per second (kIPS). We also provide both the energy for a single sample in isolation (batch=1) and steady-state energy per inference (batch=∞). Note that FINN does not provide some results for the MFC model.

TABLE II
COMPARISON OF ULEEN AGAINST FINN MODELS SFC, MFC, LFC (FPGA). FINN ROWS ARE SHADED. N/A MEANS NO DATA AVAILABLE.

Model	Latency (μs)	Xput (kIPS)	Power (W)	μJ/Inf. b=1	μJ/Inf. b=∞	LUT	BRAM	Test Acc.%
ULN-S	0.21	14,286	1.1	0.234	0.077	17,319	0	96.20
SFC	0.31	12,361	7.3	2.263	0.591	91,131	4.5	95.83
ULN-M	0.29	14,286	3.1	0.887	0.214	49,445	0	97.79
MFC	N/A	6,238	11.3	N/A	1.811	N/A	N/A	97.69
ULN-L	0.94	4070	3.4	3.148	0.826	123,117	0	98.46
LFC	2.44	1561	8.8	21.5	5.637	82,988	396	98.40

Overall, we achieve a 1.4-2.6x improvement in latency and a 1.2-2.6x improvement in throughput versus FINN, with higher accuracies in all three cases. Our improvements in energy are proportionately even stronger. We reduce energy by 6.8-9.6x for a single inference and by 6.8-8.5x for steady-state inference. These improvements clearly demonstrate that WNNs can outperform even optimized BNN implementations.

All three FINN models, as well as ULN-S and ULN-M, were implemented at 200 MHz. Due to FPGA routing congestion and consequent long wires, we were limited to implementing our large design at 85 MHz.

D. ASIC Comparison (vs Bit Fusion)

Table III and Figure 12 show the comparison between Bit Fusion and ULEEN. Bit Fusion is designed to operate with a batch size of 16; we use the same batch size for ULEEN to ensure a fair comparison. ULEEN achieves a single-inference (batch=1) latency between 0.032 μs (ULN-S) and 0.056 μs (ULN-L). The fastest Bit Fusion model (BF32) requires 838 μs, but since this model performs inference in simultaneous batches of 16, a direct comparison to single-inference latency is difficult.

Bit Fusion implements an aggressively quantized LeNet-5 model, achieving accuracy superior to ULN-L by 0.89%.

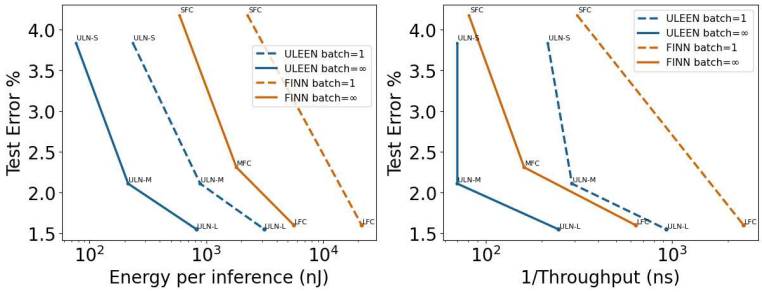


Fig. 11. Energy and Inverse Throughput vs. Error Pareto fronts for ULEEN and FINN. Both single-inference (batch=1) and steady-state (batch=∞) values are shown. Note that for batch=1, inverse throughput is equal to latency.

TABLE III
COMPARISON OF ULEEN AGAINST BIT FUSION/ LENET-5 (ASIC). BIT FUSION ROWS ARE SHADED.

Model	Xput (kIPS)	Power (W)	nJ/Inf. b=16	Area (mm^2)	Test Acc.%
ULN-S	55,556	0.84	17.5	0.61	96.20
ULN-M	55,556	2.58	57.1	2.09	97.79
ULN-L	38,462	6.23	195.5	5.22	98.46
BF8	2.0	0.26	129,731	0.60	99.35
BF16	7.1	0.81	114,914	1.59	99.35
BF32	19.1	1.79	93,589	1.65	99.35

However, ULN-L uses between **479-663x** less energy and achieves **2014-19549x** greater throughput. Models such as Bit Fusion have their place in devices such as mobile phones, where an energy budget in the hundreds of μJ is generally acceptable. However, ULEEN presents a preferable solution for extreme edge devices, or even for larger devices if a very high throughput is needed.

Note that the Bit Fusion accelerator is a hardware architecture intended to run any quantized convolutional neural networks. Here a quantized (ternary) LeNet-5 model is being run on Bit Fusion. Changing the size of the Bit Fusion accelerator (e.g. systolic array dimensions) impacts the performance of the accelerator, but not the accuracy of the model. On the other hand, with ULEEN, the accelerator and model sizes are linked. ULEEN establishes an interplay between accuracy, efficiency, and area, which can be explored depending on the application.

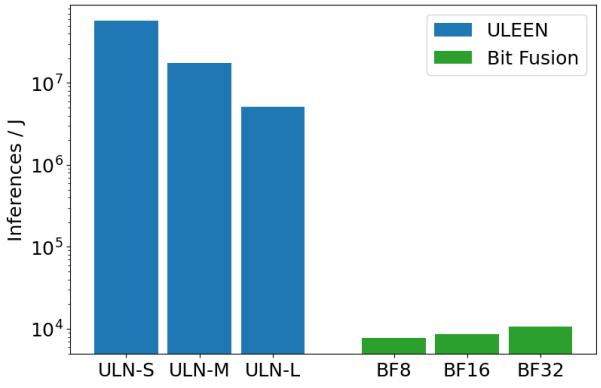


Fig. 12. Power Efficiency (Inferences per Joule) for ULEEN and Bit Fusion

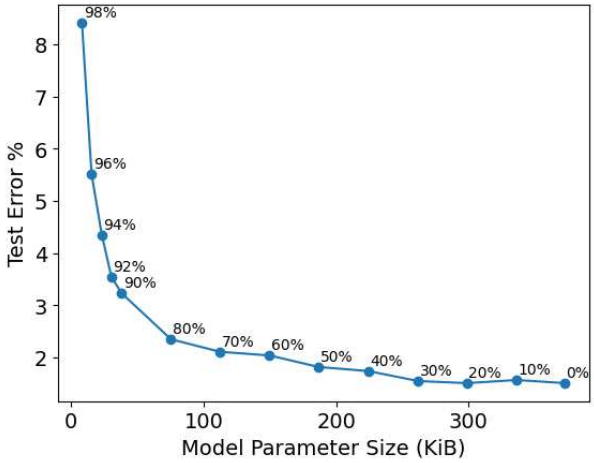


Fig. 13. Pruned Size vs. Error in ULN-L. The % value next to each point denotes the amount of pruning done.

E. Model Comparison (vs Bloom WiSARD)

Bloom WiSARD achieved accuracy comparable to large traditional WNNs at a fraction of the model size by using (binary) Bloom filters for compression [23], establishing the state of the art for efficient WNNs. We evaluate ULEEN on the same nine multi-class classification datasets used for the evaluation of Bloom WiSARD, shown in Table IV. On MNIST, we improve test accuracy from 91.5% (the previous best result on a sub-1MB WNN model) to 98.46%, achieved with ULN-L. ULEEN produces a more accurate and smaller model in all cases, with a mean 46.1% decrease in model size and 49.8% reduction in test error.

ULEEN achieves a surprising ˜99% reduction in error on the Shuttle dataset. Shuttle is an anomaly-detection dataset in which 80% of the training data belongs to the "normal" class [43]. We suspect that, since Bloom WiSARD does not incorporate bleaching, the discriminator corresponding to this class became saturated during training.

TABLE IV
ACCURACY AND MODEL SIZE OF ULEEN (ULN-L) VS. BEST PRIOR WNN WORK (BLOOM WISARD) [23].

Model Name	Test Acc.% (Bloom WiSARD)	Test Acc.% (ULEEN)	Size (KiB) (Bloom WiSARD)	Size (KiB) (ULEEN)
MNIST [36]	91.5	98.5	819	262
Ecoli [44]	79.9	87.5	3.28	0.875
Iris [45]	97.6	98.0	0.703	0.281
Letter [46]	84.8	90.0	91.3	78.0
Satimage [47]	85.1	88.0	12.7	9.00
Shuttle [43]	86.8	99.9	3.69	2.63
Vehicle [48]	66.2	76.2	4.22	2.25
Vowel [49]	87.6	90.0	6.44	3.44
Wine [50]	92.6	98.3	2.28	0.422

F. Sensitivity Analysis

1) Pruning: Figure 13 shows the model sizes and accuracies achieved by pruning ULN-L at a range of target ratios. We swept pruning in 10% increments from 0-90%, and in 2% increments from 90% to 98%. We observe almost no loss in accuracy for pruning ratios up to 30%, and only gradual

degradation up to 80%, which gives 97.65% accuracy at a 75.31 KiB model size. Accuracy deteriorates rapidly past this point, but even with 98% pruning, our test error is under 10%.

While model size decreases proportionately with the pruning ratio, the amount of hashing required does not, since different filters may be pruned from different discriminators. Therefore, in practice it is likely less efficient to use a high pruning ratio than it is to just train a smaller model.

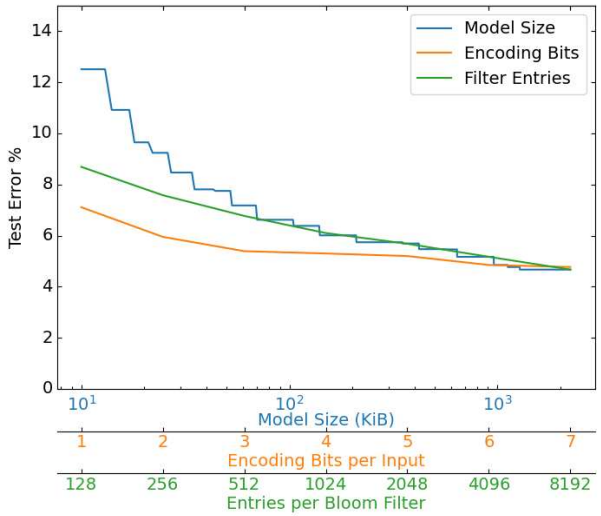


Fig. 14. Results from our one-shot sweeping runs, showing the most accurate model which was achieved with a given maximum size, the most accurate model which was achieved with a given number of encoding bits, and the most accurate model which was achieved with a given number of entries per Bloom filter.

2) One-shot Model Sweeping: Figure 14 summarizes the results from sweeping across a variety of model configurations using the one-shot training rule. We observe diminishing returns from increasing the number of thermometer encoding bits per Bloom filter and entries per Bloom filter (note that the entries per filter in the figure is on a log scale).

Accuracy for models trained with the one-shot technique increases roughly logarithmically with model size. We are unable to reach 96% accuracy with a model size under 2 MB. This demonstrates the limitations of the one-shot algorithm and the importance of incorporating feedback; we are able to reach higher accuracies with much smaller models using the multi-shot learning rule.

VI. Related Work

Our work on WNNs is suitable for edge inference applications with very low power budgets due to ULEEN's high energy efficiency. In this section, we discuss other works that target similar use cases.

Quantized DNNs: Quantization involves transforming higher precision floating-point numbers to lower precision floating-point, fixed-point or integer [51] values. Quantizing networks leads to reduced memory access costs and increases compute efficiency. DNN accelerators commonly provide hardware support for lower precisions, such as 8-bit integer in Google TPU [52] or 12-bit floating-point in Microsoft Brainwave [53]. Bit Fusion [16], which we compare to, proposes an accelerator architecture that dynamically composes low-bitwidth compute units to match the precision requirements of DNNs.

Ternary Weight Networks (TWNs) [39], [54], [55] have been proposed, where weights are constrained to +1, 0 and -1. The accuracy of TWNs has been shown to be only slightly worse than the full precision networks but significantly higher than BNNs. We compare the accuracy and efficiency of our ULN-L model against a Bit Fusion accelerator running a ternarized LeNet-5. More ASIC and FPGA implementations of ternary neural networks for MNIST are presented in [56], but these have lower accuracy, throughput and energy-efficiency than our ULEEN models.

Binary NNs: Many methods to binarize weights or activation functions in DNNs have been proposed [6], [7], [9], [10], [57]. Most modern BNN approaches first take the XNOR of the input with a learned weight vector, followed by a popcount operation. Several works have explored ASIC and FPGA accelerators for BNNs (e.g., FINN [8], FP-BNN [58], and YodaNN [59]). As illustrated in the comparison with FINN, ULEEN WNNs can achieve better latency and energy than BNNs. ReBNet [60] is another FPGA-based BNN accelerator for MNIST. Our throughput and accuracy are higher than ReBNet, but no energy numbers are provided in the paper.

Compressed DNNs: To efficiently run DNNs on devices with limited hardware resources, model compression is used [31], [61], [62], [63]. Pruning [30], [33], [64] reduces the total number of parameters, but may require retraining to avoid accuracy degradation. Weight sparsity in CNNs has been used to reduce computation complexity [65], [66], [67], [68], and bit sparsity is exploited in [69], [70]. We use two forms of compression in ULEEN: Bloom filters to reduce the size of the RAM nodes, and pruning to reduce the number of RAM nodes.

Prior WNNs: Prior work [23], [71] demonstrated that replacing the RAM nodes in WiSARD with Bloom filters improved memory efficiency and model robustness, though we are the first to use counting Bloom filters (which facilitates bleaching). We are also the first to implement a Bloom filter-based WNN in hardware, as prior work used an impractical hashing algorithm. In [72], a WiSARD-based rock-paper-scissors player was built using a ternary system, where each input digit value was uniquely associated to *rock*, *paper*, or *scissors* states. Prior memory-efficient WNNs [14], [23] report up to 91.5% accuracy on MNIST while our improvements raise accuracy to 98.46%.

Symbolist Approaches: Symbolist paradigms such as decision trees and Bayesian inference generally work well for small-scale problems. However, these models typically require sophisticated, heavy domain-dependent preprocessing, or incur large (e.g., half a million LUTs [73]) or very customized circuit implementations (e.g., clockless probabilistic p-bits [74]).

Other edge DNN acceleration approaches: Microcontroller-based approaches to edge inference, such as TinyML, have attracted a great deal of interest recently due to

their ability to use inexpensive off-the-shelf hardware [75]. However, these approaches to machine learning are thousands of times slower than dedicated accelerators. A TinyML MNIST solution on Arduino Nano using downscaled 8x8 images had ¿130,000x lower throughput and more than 32,000x higher latency than our FPGA implementation of ULN-M on 28x28 images, in addition to being less accurate [75].

VII. Conclusion

Inference on the extreme edge demands new approaches to machine learning. While existing techniques use quantized DNNs or BNNs, we propose ULEEN, an approach based on Weightless Neural Networks. ULEEN introduces and incorporates counting Bloom filters, Gaussian thermometer encoding, additive ensembles, pruning, and a novel multi-shot training algorithm to achieve superior energy and latency compared to quantized DNNs and BNNs. We present FPGA and ASIC implementations of an inference accelerator for ULEEN. Compared against the FINN FPGA-based platform for BNNs, we improve latency by 1.4-2.6x, throughput by 1.2-2.6x, and energy by 6.8-8.5x. Compared to the Bit Fusion low-precision DNN ASIC, we reduce energy by 479-663x and throughput by 2014-19549x, though with slightly lower accuracy. The energy per inference for the most accurate FINN model is 5.637 μJ whereas ULEEN consumes only 0.826 μJ per inference for the largest model presented. For the Bit Fusion model, the energy per inference ranges from 93.5-129 μJ, whereas ULEEN models only consume 0.01-0.2 μJ (i.e. more than 400x improvement), illustrating the potential of ULEEN for extreme edge devices.

The most direct opportunity for future work that we see is the development of convolutional WNNs. Convolution is important for good accuracy on larger image datasets such as ImageNet. Although support for convolution adds model and hardware complexity, we believe that WNNs have the potential to excel in this domain, as they can learn nonlinear filters, something that is not possible with traditional CNNs.

Our results show that WNNs hold substantial promise for inference on the edge. WNNs are efficient, but have historically trailed BNNs and quantized DNNs in accuracy; the improvements in ULEEN narrow or close that gap, demonstrating their suitability for ultra-low-energy and high-throughput inference.

Acknowledgement: *This research was supported in part by Semiconductor Research Corporation (SRC) Task 3015.001/3016.001 and National Science Foundation grant number 1763848. Any opinions, findings, conclusions or recommendations are those of the authors and not of the funding agencies.*

References

[1] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "Imagenet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 4 2015.

[2] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," 2021.

[3] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," 2019. [Online]. Available: https://arxiv.org/abs/1908.09791

[4] E. Covi, E. Donati, X. Liang, D. Kappel, H. Heidari, M. Payvand, and W. Wang, "Adaptive extreme edge computing for wearable devices," *Frontiers in Neuroscience*, vol. 15, 2021. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2021.611300

[5] J. Portilla, G. Mujica, J.-S. Lee, and T. Riesgo, "The extreme edge at the bottom of the internet of things: A review," *IEEE Sensors Journal*, vol. 19, no. 9, pp. 3179–3190, 2019.

[6] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, ser. NIPS'15. Cambridge, MA, USA: MIT Press, 2015, p. 3123–3131.

[7] "Binarized neural networks," *CoRR*, vol. abs/1602.02505, 2016, withdrawn. [Online]. Available: http://arxiv.org/abs/1602.02505

[8] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "Finn: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 65–74. [Online]. Available: https://doi.org/10.1145/3020078.3021744

[9] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," *CoRR*, vol. abs/1603.05279, 2016. [Online]. Available: http://arxiv.org/abs/1603.05279

[10] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," 2016.

[11] I. Aleksander, M. De Gregorio, F. França, P. Lima, and H. Morton, "A brief introduction to weightless neural systems," in *17th European Symposium on Artificial Neural Networks (ESANN)*, 04 2009, pp. 299–305.

[12] T. Ludermir, A. de Carvalho, A. Braga, and M. Souto, "Weightless neural models: A review of current and past works," *Neural Computing Surveys*, vol. 2, pp. 41–61, 01 1999.

[13] P. Yin, J. Lyu, S. Zhang, S. J. Osher, Y. Qi, and J. Xin, "Understanding straight-through estimator in training activation quantized neural nets," *CoRR*, vol. abs/1903.05662, 2019. [Online]. Available: http://arxiv.org/abs/1903.05662

[14] B. P. A. Grieco, P. M. V. Lima, M. De Gregorio, and F. M. G. França, "Producing pattern examples from "mental" images," *Neurocomput.*, vol. 73, no. 7-9, pp. 1057–1064, Mar. 2010. [Online]. Available: http://dx.doi.org/10.1016/j.neucom.2009.11.015

[15] D. Carvalho, H. Carneiro, F. França, and P. Lima, "B-bleaching : Agile overtraining avoidance in the wisard weightless neural classifier," in *ESANN*, 04 2013.

[16] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, and H. Esmaeilzadeh, "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*, 2018.

[17] R. Rohwer and M. Morciniec, "The theoretical and experimental status of the *n*-tuple classifier," *Neural Netw.*, vol. 11, no. 1, p. 1–14, Jan. 1998.

[18] I. Wickert and F. França, "Validating an unsupervised weightless perceptron," in *Proceedings of the 9th International Conference on Neural Information Processing, 2002. ICONIP '02.*, 12 2002, pp. 537 – 541 vol.2.

[19] D. O. Cardoso, D. Carvalho, D. S. F. Alves, D. F. P. de Souza, H. C. C. Carneiro, C. E. Pedreira, P. M. V. Lima, and F. M. G. França, "Financial credit analysis via a clustering weightless neural classifier," *Neurocomputing*, vol. 183, pp. 70–78, 2016.

[20] I. Aleksander, W. Thomas, and P. Bowden, "WISARD·a radical step forward in image recognition," *Sensor Review*, vol. 4, no. 3, pp. 120–124, 1984. [Online]. Available: https://www.emerald.com/insight/content/doi/10.1108/eb007637/full/html

[21] V. Vapnik and A. Chervonenkis, *On the uniform convergence of relative frequencies of events to their probabilities*. Springer, 01 2015, pp. 11–30.

[22] H. Carneiro, C. Pedreira, F. França, and P. Lima, "The exact vc dimension of the wisard n-tuple classifier," *Neural Computation*, pp. 1–32, 11 2018.

[23] L. S. de Araújo, L. D. Verona, F. M. Rangel, F. F. de Faria, D. S. Menasché, W. Caarls, M. Breternitz, S. Kundu, P. M. V. Lima, and

F. M. G. França, "Memory efficient weightless neural network using bloom filter," in *ESANN*, 2019.

[24] J. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0022000079900448

[25] A. Appleby, "Murmurhash3," https://github.com/aappleby/smhasher, 2016.

[26] A. Kappaun, K. Camargo, F. Rangel, F. Firmino, P. M. V. Lima, and J. Oliveira, "Evaluating binary encoding techniques for wisard," in *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*, 2016, pp. 103–108.

[27] P. Xavier, M. D. Gregorio, F. M. G. França, and P. M. V. Lima, "Detection of elementary particles with the wisard n-tuple classifier," in *ESANN*, 2020.

[28] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 1–15.

[29] L. A. Lusquino Filho, L. F. Oliveira, A. L. Filho, G. P. Guarisa, L. M. Felix, P. M. Lima, and F. M. França, "Extending the weightless wisard classifier for regression," *Neurocomputing*, vol. 416, pp. 280–291, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S092523122030504X

[30] S. Han, H. Mao, and W. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *International Conference on Learning Representations (ICLR)*, 10 2016.

[31] J. Yu, A. Lukefahr, D. Palframan, G. Dasika, R. Das, and S. Mahlke, "Scalpel: Customizing dnn pruning to the underlying hardware parallelism," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 548–560.

[32] T. Zhang, S. Ye, K. Zhang, J. Tang, W. Wen, M. Fardad, and Y. Wang, "A systematic dnn weight pruning framework using alternating direction method of multipliers," in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer International Publishing, 2018, pp. 191–207.

[33] W. Niu, X. Ma, S. Lin, S. Wang, X. Qian, X. Lin, Y. Wang, and B. Ren, *PatDNN: Achieving Real-Time DNN Execution on Mobile Devices with Pattern-Based Weight Pruning*. New York, NY, USA: Association for Computing Machinery, 2020, p. 907–922. [Online]. Available: https://doi.org/10.1145/3373376.3378534

[34] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations*, 12 2014.

[35] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, p. 1929–1958, jan 2014.

[36] Y. LeCun and C. Cortes, "MNIST handwritten digit database," http://yann.lecun.com/exdb/mnist/, 2010. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[37] Xilinx. (2021) FINN: Dataflow compiler for QNN inference on FPGAs. [Online]. Available: https://xilinx.github.io/finn/

[38] G. I. o. T. ACT Lab. (2018) Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating DNNs. [Online]. Available: http://www.act-lab.org/artifacts/bitfusion/

[39] F. Li and B. Liu, "Ternary weight networks," *CoRR*, vol. abs/1605.04711, 2016. [Online]. Available: http://arxiv.org/abs/1605.04711

[40] NCSU. (2018) FreePDK45. [Online]. Available: https://www.eda.ncsu.edu/wiki/FreePDK45:Contents

[41] H. Packard. (2017) Cacti. [Online]. Available: https://github.com/HewlettPackard/cacti/

[42] M. Bayer, "Mako templates for python," 2021. [Online]. Available: https://www.makotemplates.org/

[43] J. Catlett, "Statlog (shuttle) data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Statlog+(Shuttle)

[44] K. Nakai, "Ecoli data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/ecoli

[45] R. Fisher, "Iris data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/iris

[46] D. J. Slate, "Letter recognition data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/letter+recognition

[47] A. Srinivasan, "Statlog (landsat satellite) data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Statlog+(Landsat+Satellite)

[48] P. Mowforth and B. Shepherd, "Statlog (vehicle silhouettes) data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Statlog+(Vehicle+Silhouettes)

[49] D. Deterding, M. Niranjan, and T. Robinson, "Connectionist bench (vowel recognition - deterding data) data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Vowel+Recognition+-+Deterding+Data)

[50] S. Aeberhard, "Wine data set." [Online]. Available: https://archive.ics.uci.edu/ml/datasets/wine

[51] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017. [Online]. Available: https://arxiv.org/abs/1712.05877

[52] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. luc Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, and J. Ross, "In-datacenter performance analysis of a tensor processing unit," 2017. [Online]. Available: https://arxiv.org/pdf/1704.04760.pdf

[53] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A Configurable Cloud-Scale DNN Processor for Real-Time AI," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 1–14.

[54] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Trained ternary quantization," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. [Online]. Available: https://openreview.net/forum?id=S1_pAu9xl

[55] Z. Lin, M. Courbariaux, R. Memisevic, and Y. Bengio, "Neural networks with few multiplications," *CoRR*, vol. abs/1510.03009, 2016.

[56] H. Alemdar, V. Leroy, A. Prost-Boucle, and F. Pétrot, "Ternary neural networks for resource-efficient ai applications," in *2017 International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 2547–2554.

[57] J. Fromm, M. Cowan, M. Philipose, L. Ceze, and S. Patel, "Riptide: Fast end-to-end binarized neural networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2, 2020, pp. 379–389. [Online]. Available: https://proceedings.mlsys.org/paper/2020/file/2a79ea27c279e471f4d180b08d62b00a-Paper.pdf

[58] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "Fp-bnn: Binarized neural network on fpga," *Neurocomputing*, vol. 275, pp. 1072–1086, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231217315655

[59] R. Andri, L. Cavigelli, D. Rossi, and L. Benini, "Yodann: An architecture for ultralow power binary-weight cnn acceleration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 48–60, 2018.

[60] M. Ghasemzadeh, M. Samragh, and F. Koushanfar, "Rebnet: Residual binarized neural network," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2018, pp. 57–64.

[61] J. H. Ko, D. Kim, T. Na, J. Kung, and S. Mukhopadhyay, "Adaptive weight compression for memory-efficient neural networks," in *Design, Automation Test in Europe Conference Exhibition (DATE), 2017*, 2017, pp. 199–204.

[62] C. Ding, S. Liao, Y. Wang, Z. Li, N. Liu, Y. Zhuo, C. Wang, X. Qian, Y. Bai, G. Yuan, X. Ma, Y. Zhang, J. Tang, Q. Qiu, X. Lin, and B. Yuan, "Circnn: Accelerating and compressing deep neural networks using block-circulant weight matrices," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 395–408.

[63] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, "On-demand deep model compression for mobile devices: A usage-driven model selection framework," in *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 389–400. [Online]. Available: https://doi.org/10.1145/3210240.3210337

[64] F. N. Iandola, M. W. Moskewicz, K. Ashraf, S. Han, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and ¡1mb model size," *ArXiv*, vol. abs/1602.07360, 2016.

[65] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 1–13.

[66] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 27–40.

[67] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen, "Cambricon-x: An accelerator for sparse neural networks," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[68] X. Zhou, Z. Du, Q. Guo, S. Liu, C. Liu, C. Wang, X. Zhou, L. Li, T. Chen, and Y. Chen, "Cambricon-s: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 15–28.

[69] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 382–394.

[70] M. Mahmoud, K. Siu, and A. Moshovos, "Diffy: a déjà vu-free differential deep neural network accelerator," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 134–147.

[71] L. Santiago, L. Verona, F. Rangel, F. Firmino, D. S. Menasché, W. Caarls, M. Breternitz Jr, S. Kundu, P. M. Lima, and F. M. França, "Weightless neural networks as memory segmented bloom filters," *Neurocomputing*, vol. 416, pp. 292–304, 2020.

[72] D. F. d. Souza, H. C. Carneiro, F. M. França, and P. M. Lima, "Rock-paper-scissors wisard," in *2013 BRICS Congress on Computational Intelligence and 11th Brazilian Congress on Computational Intelligence*, 2013, pp. 178–182.

[73] R. Choudhury, S. R. Ahamed, and P. Guha, "Efficient hardware implementation of decision tree training accelerator," in *https://ieeexplore.ieee.org/document/942607345*.

[74] R. Faria, J. Kaiser, K. Y. Camsari, and S. Datta, "Hardware design for autonomous bayesian networks," *Frontiers in Computational Neuroscience*, vol. 15, 2021. [Online]. Available: https://www.frontiersin.org/article/10.3389/fncom.2021.584797

[75] Simone, "TinyML or Arduino and STM32: Convolutional Neural Network (CNN) Example, Accessed Nov 22, 2021," 2020. [Online]. Available: https://eloquentarduino.github.io/2020/11/tinyml-on-arduino-and-stm32-cnn-convolutional-neural-network-example/